Autonomous Ackermann Vehicle System: Simulation, Control, Planning, Localization, and Hardware Integration

Team 24
MCTR1002 - Autonomous Systems, Mechatronics Department
German University in Cairo

***Abstract*—This report summarizes the complete development of an autonomous Ackermann-like vehicle system from the first validation stage to the final integrated racing configuration. The project was divided into five milestones. The early work focused on setting up Ubuntu, ROS 2 Jazzy, Gazebo, the Raspberry Pi environment, and a reusable project package. The second stage introduced open-loop response testing and keyboard teleoperation for both simulation and hardware. The third stage converted the vehicle from manual/open-loop operation to closed-loop longitudinal and lateral control. The fourth stage added track construction, path planning, lane selection, obstacle avoidance, and hardware localization through onboard sensors. The fifth stage integrated the vehicle model, controller, planner, and localization/filtering node in one launch architecture. The final system follows a desired speed profile, maintains or changes lanes according to the track scenario, estimates vehicle states, and sends speed and steering commands to the physical vehicle through the embedded platform.**

***Index Terms*—ROS 2, Gazebo, Ackermann vehicle, autonomous systems, longitudinal control, lateral control, Stanley controller, localization, Kalman filter, Raspberry Pi, Arduino.**

I. Introduction

The project target was to build a complete autonomous driving stack for a small-scale Ackermann vehicle. The work combined simulation and hardware implementation. In simulation, the vehicle was modeled in Gazebo and controlled through ROS nodes. In hardware, a fabricated scaled vehicle was equipped with a Raspberry Pi, Arduino-based low-level actuation, steering servo, drive motor, and sensor interfaces. The final objective was not only to move the vehicle, but to connect all autonomous modules: planning, control, localization, filtering, and system launch integration.

The development followed a progressive engineering process. First, the ROS environment and package structure were validated. Second, the vehicle was driven in an empty world using constant commands and teleoperation. Third, feedback controllers were added to regulate speed and lateral position. Fourth, Gazebo worlds representing the empty track, racing track, and city track were built, and the planner generated velocity and lane references. Finally, the localization module added noise and filtering in simulation and interfaced with real sensor readings on hardware.

II. System Architecture

The final architecture is based on ROS communication between modular nodes. Each node has a specific responsibility, which made the design easier to debug and reuse across milestones. The planner publishes desired speed and desired lane. The controller subscribes to these references and to the estimated vehicle states. The localization node publishes filtered position, heading, and velocity. The vehicle model or Arduino interface subscribes to the final actuation commands.

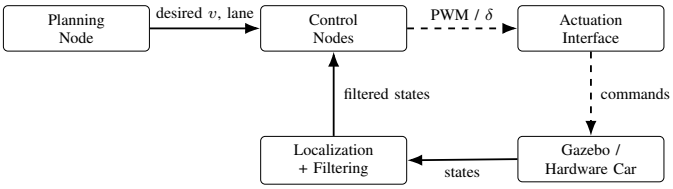

Fig. 1. Final ROS architecture connecting vehicle states, localization, planning, control, and actuation.

The global control vector was treated as

$$u = [v_d, \delta]^T, \tag{1}$$

where v_d is the desired longitudinal speed and δ is the steering command. The main estimated state vector used by the higher-level nodes was

$$x = [x_{car}, y_{car}, \theta_{car}, v_{car}]^T. \tag{2}$$

This separation allowed the same controller and planner logic to operate with either ideal Gazebo states or filtered sensor-based states.

III. ROS Package Organization and Communication Design

The package was organized so that each milestone added functionality without replacing the previous work. Source files were placed in the `src` directory, launch files were placed in the `launch` directory, and Gazebo world files were placed in the `Worlds` directory. This organization made it possible to test one subsystem at a time while still keeping a single final package for integration.

The topic interface was kept consistent across the project. The planner did not publish motor PWM directly. Instead, it published references. The controller converted references into actuation commands. The localization node did not decide the path; it only estimated the state. This separation is important

TABLE I
MAIN ROS NODES AND THEIR PROJECT ROLES.

Node / Module	Main Inputs	Main Outputs and Role
Validation node	ROS execution environment	Prints a validation message to confirm that the package, dependencies, and runtime setup are correct.
Open-loop response node	Constant desired speed and steering parameters	Publishes fixed commands to the vehicle for identifying motion direction, command sign, and basic vehicle response.
Teleoperation node	Keyboard arrow keys	Converts manual keyboard inputs into speed and steering commands for simulation and hardware tests.
Speed control node	Desired speed and actual speed	Computes the longitudinal control effort and reduces speed tracking error.
Lateral control node	Desired lane, position, heading, and speed	Computes steering angle using lateral and heading errors to keep or change lanes.
Planning node	Track type, vehicle position, obstacle locations	Publishes desired speed profile and desired lane according to the selected driving scenario.
Localization/filtering node	Gazebo states or hardware sensor readings	Publishes filtered vehicle states for controller and planner feedback.
Arduino interface	Desired speed/steering from Raspberry Pi	Applies PWM and steering commands and returns encoder/sensor measurements.

in autonomous systems because each module can be tuned or replaced independently.

The launch files were also important because they stored experiment settings. Initial position, initial heading, desired speed, desired lane, map selection, and noise parameters were all passed through launch arguments or ROS parameters. This reduced manual errors during testing and allowed repeated experiments under the same conditions.

IV. MILESTONE 1: ENVIRONMENT, ROS VALIDATION, AND VEHICLE SELECTION

The first milestone established the basic software and hardware environment. Ubuntu 24.04 and ROS 2 Jazzy were installed on the development laptop, while the embedded side used a Raspberry Pi prepared for ROS communication. The project package was created using the required naming convention and was used as the main workspace for later nodes, launch files, worlds, and hardware communication scripts.

A validation node was created to confirm that the ROS installation and package build system worked correctly. This node printed a validation message and proved that the package could be built, sourced, and executed. This step was important because later milestones depended on a stable ROS workspace, correct dependencies, and repeatable launch behavior.

The simulation vehicle was selected as an Ackermann-like car model because the physical platform also uses front steering and rear/drive-wheel propulsion. During this stage, the vehicle topics were inspected to identify the publishers and subscribers needed for motion control. The most important subscribed topics were the steering and drive command topics, while the most important published topics were state-related topics such as velocity, position, heading, and steering feedback. This mapping became the interface contract for the open-loop, teleoperation, closed-loop, and final integrated nodes.

TABLE II
IMPLEMENTED DRIVING MODES DURING THE EARLY PROJECT STAGES.

Mode	Function
Validation	Confirms ROS package execution and terminal output.
Open-loop response	Sends fixed speed and steering commands for basic maneuver testing.
Teleoperation	Converts keyboard arrows into speed and steering references.
Closed-loop control	Uses feedback to reduce speed and lateral tracking errors.
Autonomous mode	Uses planner, controller, localization, and launch integration.

V. MILESTONE 2: OPEN-LOOP RESPONSE AND TELEOPERATION

The second milestone moved the system from setup validation to actual vehicle motion. Two driving modes were implemented: open-loop response and teleoperation. In the open-loop response mode, the node published constant speed and steering commands. The purpose was to check the vehicle response without feedback compensation. Different command pairs were tested to produce forward motion, backward motion, right steering, and left steering. This helped verify topic names, command signs, actuator limits, and the physical feasibility of the car's steering mechanism.

The teleoperation node added keyboard-based driving. The up and down arrows were mapped to acceleration and deceleration, while the right and left arrows were mapped to positive and negative steering. This mode was useful for manual testing because it allowed fast validation of the Gazebo vehicle and the hardware platform before the closed-loop controller was ready. It also gave a simple way to observe how steering angle and speed interact in the Ackermann chassis.

On the hardware side, the chassis was assembled and the first motor tests were performed using Arduino code with fixed PWM. The goal was to confirm the wiring, motor driver behavior, battery supply, and steering servo operation

before adding sensors or feedback. USB serial communication between Raspberry Pi and Arduino was prepared so that the ROS teleoperation node could send desired speed and steering values to the microcontroller. The physical vehicle views used during these tests are shown in Fig. 2.

VI. Milestone 3: Closed-Loop Longitudinal and Lateral Control

The third milestone introduced feedback control. The vehicle was no longer treated as a system that only receives fixed commands; instead, the controller compared desired and actual states and corrected the command accordingly. Two control nodes were implemented: one for speed control and one for lateral/lane control.

For longitudinal control, the controller subscribed to the actual vehicle speed and read the desired speed from ROS parameters or planner output. A proportional control law was used as the basic structure:

$$e_v(t) = v_{ref}(t) - v(t), \tag{3}$$

$$u_v(t) = K_{p,v} e_v(t), \tag{4}$$

where e_v is the speed error and $K_{p,v}$ is the longitudinal proportional gain. In implementation, the command was limited using saturation to avoid sending unsafe values to the motor driver or simulated actuator.

For lateral control, the objective was to reduce the error between the actual lateral position and the desired lane center. A PID or Stanley-style approach can be used for this task. In our design, the lateral controller used the cross-track error and heading error to generate the steering command. The general Stanley form is

$$\delta = \theta_e + \tan^{-1}\left(\frac{k e_y}{v + \epsilon}\right), \tag{5}$$

where θ_e is heading error, e_y is lateral error, k is the lateral gain, v is vehicle speed, and ϵ avoids division by zero at low speed. This controller is suitable for lane tracking because it corrects both orientation and lateral displacement.

The closed-loop launch file placed the vehicle at specified initial conditions $[x_{veh}, y_{veh}, \theta_{veh}]$ and passed the desired speed and lane through ROS parameters. Several tests were performed with different initial positions, desired speeds, and target lanes to check whether the car converged to the requested lane without excessive oscillation. On hardware, a simple P controller was used for the drive motor PWM based on encoder feedback.

A. Controller Tuning Procedure

Controller tuning was performed gradually. For speed control, the proportional gain was increased until the vehicle reached the desired speed with acceptable rise time and without strong oscillation. If the gain was too small, the vehicle reacted slowly and kept a visible steady-state error. If the gain was too large, the command became aggressive and caused oscillatory acceleration. Therefore, command saturation was added to keep the output within the safe PWM or simulated command range.

TABLE III
Typical signals checked during controller debugging.

Signal	Reason for checking
v_{ref} and v	Confirms speed tracking and detects overshoot or slow response.
y_{ref} and y	Confirms lane tracking and lane-change convergence.
θ	Shows whether the car heading is aligned with the path.
δ	Confirms steering saturation and oscillation behavior.
PWM command	Confirms that motor commands remain within safe limits.

For lateral control, tuning was more sensitive because steering affects both heading and lateral displacement. A small lateral gain made the car converge slowly to the desired lane. A large gain caused sharp steering and possible overshoot around the lane center. The final tuning balanced lane convergence and smooth steering. During lane-change tests, the vehicle was expected to move from one lane center to the other without crossing track boundaries or hitting obstacles.

VII. Milestone 4: Track Worlds, Planning, and Hardware Localization

The fourth milestone extended the system from an empty world to structured driving scenarios. Three Gazebo worlds were prepared: an empty straight track, a racing track with obstacles, and a city track with curvatures. The empty track was used to verify longitudinal motion and lane keeping. The racing track was used to test obstacle avoidance through lane changing. The city track was used to test curvature handling and continuous path following.

The racing scenario used two lane centers. Lane 1 was represented by $y = 0.1875$ m, and Lane 2 was represented by $y = -0.1875$ m. The planner selected the lane according to the longitudinal position of the car and the known obstacle locations. Before reaching the first obstacle around $x = 4$ m, the planner commanded a lane change from Lane 1 to Lane 2. Before reaching the second obstacle around $x = 8$ m, it commanded a lane change back to Lane 1. During lane changes and curvatures, the desired velocity was reduced to improve stability, then increased again after the maneuver.

VIII. Results and Discussion

The project produced a complete modular autonomous system. In the first stages, the vehicle could be launched and controlled in an empty world. The open-loop response verified the basic command interface, but it was not enough for accurate tracking because any disturbance or model mismatch caused steady-state error. Teleoperation was useful for debugging because it allowed immediate manual control of speed and steering.

After adding closed-loop control, the vehicle behavior became more stable. The speed controller reduced the difference

Fig. 2. Physical Ackermann vehicle platform views used for hardware testing.

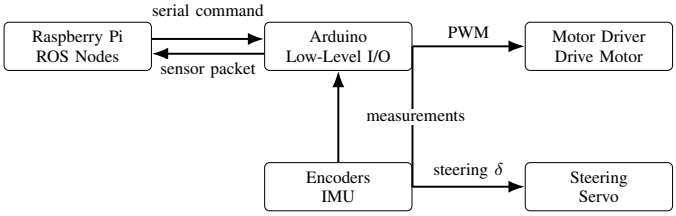

Fig. 3. Hardware communication and actuation flow between Raspberry Pi, Arduino, sensors, and actuators.

between desired and actual velocity, while the lateral controller drove the vehicle toward the desired lane. The planner then converted the control system into a higher-level autonomous system by changing the desired lane and speed depending on track conditions. On the racing track, the desired lane changed before obstacles, while on the city track, velocity reduction was used around curves.

The localization node improved the quality of the state information sent to the controller. In simulation, adding Gaussian noise showed why filtering is required: raw measurements can produce noisy steering and speed commands. After filtering, the estimated states became smoother, which should reduce oscillations in the control action. In hardware, encoder and IMU-based localization provided the minimum required feedback for closed-loop motion.

Some limitations remain. Wheel odometry can drift because it depends on tire contact, wheel radius calibration, and heading accuracy. The lateral controller also depends strongly on tuning the Stanley/PID gains. At low speeds, steering commands can become sensitive, so saturation and the small ϵ term are required. On the hardware side, cable management, power stability, and sensor mounting quality directly affect reliability.

IX. Engineering Challenges

Several practical challenges appeared during the project. In simulation, topic names and message types had to be verified carefully because a wrong topic connection makes the controller appear correct in code while the vehicle receives no command. Launch files also required correct parameter names, because the same node behavior changes depending on the supplied desired speed, lane, and initial state.

In control, the largest challenge was the coupling between longitudinal and lateral behavior. At higher speed, the same steering angle produces a stronger lateral motion, so the lateral controller must be tuned together with the speed profile. This is why the planner reduces the reference speed during lane changes and curvatures. In localization, raw encoder readings and IMU heading can contain noise, offset, or drift. The filtering stage was therefore necessary to stabilize the feedback signal before using it in the controller.

On hardware, mechanical alignment and wiring reliability were as important as the code. The steering linkage must be symmetric enough for left and right commands to produce predictable turns. The battery must supply enough current for the drive motor without resetting the electronics. The Raspberry Pi, Arduino, and motor driver also need a common ground reference to avoid communication and actuation problems.

X. Conclusion

The project successfully progressed from ROS installation and validation to a complete autonomous Ackermann vehicle architecture. The implemented work included ROS package creation, Gazebo vehicle launching, open-loop driving, keyboard teleoperation, longitudinal speed control, lateral lane control, Gazebo track worlds, planning for obstacle avoidance, wheel-odometry localization, sensor communication, Kalman filtering, and final system integration. The final architecture is modular and suitable for both simulation and hardware because each node communicates through clear ROS topics. Future improvements should focus on better sensor fusion, more accurate calibration of wheel odometry, smoother trajectory generation, and systematic gain tuning for the speed and lateral controllers.

References

[1] Open Robotics, “ROS 2 Documentation,” available: https://docs.ros.org/.

[2] Open Robotics, “Gazebo Simulator Documentation,” available: https://gazebosim.org/docs.

[3] S. Thrun et al., “Stanley: The Robot that Won the DARPA Grand Challenge,” Journal of Field Robotics, 2006.

[4] R. E. Kalman, “A New Approach to Linear Filtering and Prediction Problems,” Transactions of the ASME–Journal of Basic Engineering, 1960.

[5] R. Rajamani, *Vehicle Dynamics and Control*. Springer, 2012.